GSoC 2026: AutoEIT

Candidate: Leonardo Santoro

Project: AutoEIT (Automated Electrical Impedance Tomography)

Organization: HumanAI

Task: Test I

This notebook implements an automated ASR pipeline for the AutoEIT task. By utilizing voice activity detection and custom heuristic filtering, it produces high-accuracy, verbatim transcripts that strictly preserve participant disfluencies, hesitations, and original syntax while mapping results directly to structured datasets.

Firstly, I installed the necessary AI libraries (openai-whisper for transcription and jiwer for accuracy metrics).

```
!pip install openai-whisper jiwer
```

Show hidden output

Then I established a secure connection to Google Drive, allowing the notebook to directly access and process my stored audio files and Excel templates.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

PRE-PROCESSING AND SEGMENTING THE AUDIO:

- The system handles inconsistent recording starts by applying variable offsets (e.g., 140s or 720s) to skip non-relevant introductory audio.
- I used `librosa` with a 24dB relative threshold, the system isolates speech from background noise.
- To provide Whisper with enough context, individual sound bites are merged into "sentences" if the silence between them is less than 3 seconds

HIGH PRECISION TRANSCRITP:

- I utilized the Whisper Medium model on CUDA (GPU) for an optimal balance between processing speed and linguistic accuracy.
- A custom initial_prompt containing common Spanish fillers ("Eh... bueno... ah...") is used to make the AI include disfluencies rather than cleaning them up.
- By setting temperature: 0.0 and beam_size: 5, I eliminated randomness, ensuring the most mathematically probable transcription is chosen every time.

ADVANCED ERROR CORRECTION AND LOGIC:

- Silence gaps >= 15 seconds are interpreted as skipped questions and automatically logged as [NO RESPONSE].
- If a sentence is very short or lacks punctuation, the system "bridges" it to the next segment (up to a 4-second gap) to prevent fragmented thoughts.
- A custom filter detects and removes "stutter-loops" (where the AI repeats the same long phrase twice) by analyzing word-pattern symmetry.
- A blacklist filters out common AI "ghost" words (e.g., "suscríbete", "like", "amara") that often appear during low-quality audio or static.

```
import whisper
import librosa
import numpy as np
import pandas as pd
import os
import torch

# Paths and audio files
FOLDER_PATH = "/content/drive/MyDrive/AutoEIT_Project/"
EXCEL_FILE = FOLDER_PATH + "AutoEIT_Sample.xlsx"
FINAL_OUTPUT = FOLDER_PATH + "AutoEIT_Final_Submission_38010.xlsx"
AUDIO_FILES = [
    FOLDER_PATH + "038010_EIT-2A.mp3",
    FOLDER_PATH + "038011_EIT-1A.mp3",
    FOLDER_PATH + "038012_EIT-2A.mp3",
    FOLDER_PATH + "038015_EIT-1A.mp3"
```

```
]
SHEET_NAMES = ["38010-2A", "38011-1A", "38012-2A", "38015-1A"]

# I chose to load the Medium version of the Whisper model into memory
# Using device="cuda" ensures we use the GPU
model = whisper.load_model("medium", device="cuda")


# |------- TRANSCRIPTION FUNCTION -------|

def AudioFileTranscription(AUDIO_FILE, offset_val):
    print(f"\n Loading: {os.path.basename(AUDIO_FILE)} (offset = {offset_val}s)...")

    # audio = an array of decimals, where each number is a single "snapshot" of the sound wave at a specific micro-moment
    # sr = how many of those numbers in audio equal to 1s in real-time sound
    audio, sr = librosa.load(AUDIO_FILE, sr=16000, offset=offset_val)   # Whisper was trained on 16,000 Hz audio

    # Let's split audio in intervals, where every sound 24 decibels below the peak volume of the file is considered silence
    intervals = librosa.effects.split(audio, top_db=24)
    grouped_intervals = []   # we will need it to create sentences
    # Only cut the audio if the silence lasts longer than 3 seconds (3 * 1600 samples)
    min_silence_frames = 3.0 * sr

    # Let's fill grouped_intervls with all the sentences (according to our rules)
    for interval in intervals:
        start, end = int(interval[0]), int(interval[1])
        if not grouped_intervals:
            grouped_intervals.append([start, end])
        else:
            last_interval = grouped_intervals[-1]
            if (start - last_interval[1]) < min_silence_frames:
                last_interval[1] = end
            else:
                grouped_intervals.append([start, end])

    # We will need these options later
    options = {
        "language": "es",  # stay in Spanish language
        "task": "transcribe", # listen Spanish and write Spanish (DON'T translate)
        "condition_on_previous_text": False, # if it was True, Whisper would have tried to use what was said in the previou
        "beam_size": 5, # these are our 5 most likely sentences, each one with its own probability to be the right one
        "temperature": 0.0, # the model needs to be deterministic = only choose the most probable word, without trying to g
        "initial_prompt": "Eh... bueno, yo... ah, sí, fui a la... umm... a la tienda."  # here we're giving a hint of what
                                                                                         # this helps the model understand t
    }

    transcriptions = []
    last_end_time = None
    # Sometimes Whisper, especially after a long pause, starts allucinating and hear things like "Subscribe", "Gracias", "l
    # We can detect these words and not consider them as a word the student actually said
    ghosts = ["amara", "gracias", "vídeo", "video", "suscríbete", "like","no, no"]


    i = 0
    while i < len(grouped_intervals):
        start, end = grouped_intervals[i]

        # If the student doesn't talk for 15 seconds straight, we can assume he skipped an entire sentence
        if last_end_time is not None:
            gap_seconds = (start - last_end_time) / sr
            if gap_seconds >= 15.0:
                skipped_questions = int(gap_seconds // 15)
                for _ in range(skipped_questions):
                    if len(transcriptions) < 30:
                        transcriptions.append("[NO RESPONSE]")

        # Let's slice this specific sound (a sentence) in chunk, and hand it to the Whisper Medium model, to transcribe it
        chunk = audio[start:end]
        result = model.transcribe(chunk, **options)  # this is a dictionary containing the text, timestamps, and confidence
        text = result["text"].strip()   # and let's just take the text out of this dictionary


        # !----- Now we're going to manage how we deal with repetitions, short sentences, "..." and ghosts -----!


        # ---- REPETITION ----
        # If the text is long and looks like a double-repeat, we trim it (eg: "The cat is on the table. The cat is on the t
        words = text.split()
        if len(words) >= 4:
            mid = len(words) // 2
            # Check if the first half is the same as the second half
            if words[:mid] == words[mid:]:
```

```
                text = " ".join(words[:mid])
                print(f"Trimmed a repetition in row {len(transcriptions)+1}")


            # ---- SHORT SENTENCE BRIDGE ----
            # If the text is just one word (like "Fue."), it's probably the end of the previous sentence that Whisper split.
            is_short_continuation = len(words) <= 1 and len(transcriptions) > 0

            if is_short_continuation and (i > 0):
                # We can decied wether to merge this word into the LAST transcription, or just forget it
                print(f"Merging short sentence: '{text}' into row {len(transcriptions)}")
                # I noticed the student never pronounced "Fue." in the first place, so I decided to forget it (by commenting th
                # transcriptions[-1] = transcriptions[-1] + " " + text

                # Now we don't increment the row count, and just move to the next audio segment
                last_end_time = end
                i += 1
                continue


            # ---- LINKING "..." (4s Rule) ----
            # If no period/punctuation OR ends in dots OR is very short...
            is_unfinished = not text.endswith(('.', '!', '?')) or text.endswith('...')
            is_short = len(text.split()) <= 2

            if (is_unfinished or is_short) and (i + 1 < len(grouped_intervals)):
                next_start, next_end = grouped_intervals[i+1]  # this is the next interval
                gap_to_next = (next_start - end) / sr  # this is the silence time between the current and the next interval

                # ... bridge the gap if silence is less than 4 seconds
                if gap_to_next < 4.0:
                    print(f"Merging: '{text}' (Gap: {gap_to_next:.1f}s)")
                    grouped_intervals[i][1] = next_end  # it takes the next interval and glues it to the current one
                    grouped_intervals.pop(i+1)  # it removes the next interval, since we just glued it to the current one
                    continue

            # ---- GHOSTS ----
            if text and not any(f in text.lower() for f in ghosts): # only append it to the transcriptions if it's not a ghost
                transcriptions.append(text)
                print(f"Row {len(transcriptions)}: {text}")
                last_end_time = end

            i += 1

        return transcriptions
```

DATA INTEGRATION AND EXPORT:

- Results are automatically mapped back to a specific Excel Template, aligning the transcription exactly with the original *Sentence_ID* and *Stimulus*.
- A try-except block ensures that if a participant's sheet is missing, a fallback dataframe is generated to prevent pipeline failure.
- The system caps processing at 30 items per participant to maintain alignment with the AutoEIT task structure.

```
    # |------- MAIN EXECUTION LOOP -------|
    with pd.ExcelWriter(FINAL_OUTPUT, engine='openpyxl') as writer:
        for i in range(len(AUDIO_FILES)):
            current_id = SHEET_NAMES[i]

            print(f"\n STARTING: {current_id}")

            # Read the ORIGINAL template
            try:
                original_df = pd.read_excel(EXCEL_FILE, sheet_name=current_id)
                original_df['Transcription'] = original_df['Transcription'].astype(object) # I inserted this to fix a Pandas Dt
            except Exception as e:
                # If the sheet is not present, instead of crashing, let's create a new empty spreadsheet
                print(f"Warning: Could not find sheet {current_id}. Creating fallback.")
                original_df = pd.DataFrame(columns=['Sentence_ID', 'Stimulus', 'Transcription'])

            # Run the TRANSCRIPTION
            current_offset = 720 if i == 2 else 140  # The third sheet's audio has as 12 min delay = 720 seconds, while the oth
                                                     # However, I noticed that the last sheet's audio actually starts right aft
            all_text = AudioFileTranscription(AUDIO_FILES[i], current_offset) #list of string with all the transcribed sentence

            # Let's insert the results in Column C (Index 2), capped at 30 rows
            for j in range(min(len(original_df), len(all_text), 30)):  # it will either finish before the 30 lines, or at line
                original_df.iloc[j, 2] = all_text[j]

            # Let's save the results in a sheet
            original_df.to_excel(writer, sheet_name=current_id, index=False)
```

```
        print(f"Finished {current_id}, with {min(len(all_text), 30)} rows.")


print(f"\nEVERYTHING HAS BEEN PROCESSED!")
print(f"Saved the final file at: {FINAL_OUTPUT}")
```

```
Row 5: ¿Quién hizo sé qué pasero es?
Row 6: Y turo, seca, lezard... también.
Row 7: La calle se... se... eh... mmm... Los calles...
Row 8: Pueblo, y aquí, se que ella...
Row 9: Las casas son bonitas, pero caras.
Row 10: Me gustan las paléculas, pecar, miel...
Row 11: El chico es un bonalio... es español.
Row 12: Después, llegar, dormir...
Row 13: Quiero la casa, esta, a mis animales.
Row 14: A nosotros, fascinabas a los... nosotros.
Row 15: Es... el... conversinos... del...
Row 16: Me gustaría que los besos estés... ya.
Row 17: Closar a la derecha. Umm... Mmm...
Row 18: He terminado después del... lamento.
Row 19: Me gustaría hacer pinatas... esta... ...gustaria...
Row 20: El niño... el niño Sirio Gatto es tan muy triste.
Row 21: Una amiga picado en mi...
Row 22: El gato sin... sin el negro y el perro.
Merging short sentence: 'Ah...' into row 22
Row 23: Que es limpiar tus cuartos.
Row 24: La cantonalidad es afortunada a las personas.
Row 25: Después de llegar a mi casa, voy a la cena.
Row 26: La policía era foco, so...
Row 27: Le pide... eh... una... mega...
Row 28: El... tarea es tan no difícil para darles.
Row 29: ¿Qué está en la mesa? ¿Este libro?
Row 30: Hay muchos ejemplos, pero...
Finished 38012-2A, with 30 rows.

 STARTING: 38015-1A

 Loading: 038015_EIT-1A.mp3 (offset = 140s)...
Row 1: Quiero encontrarme a pelo.
Row 2: El libro está en la mesa.
Row 3: El carro no tiene pelo.
Row 4: Él se ducha cada mañana.
Row 5: Que dice usted es que va a ser hoy.
Row 6: ¿Tudo que sepa manajar bien?
Row 7: Las calles de esta ciudad son muy anchas.
Row 8: Puede que lleve mañana toro al día.
Row 9: Las casas son muy bonitas, pero caras.
Row 10: Me gustan las películas que acaban bien.
Row 11: El chico con esto algo es español.
Row 12: Después de cenar, me fui a dormir tranquilo.
Row 13: Quiero una casa en donde vivan mis animales.
Row 14: A nosotros nos fascinan las fiestas grandes.
Row 15: Ella solo bebe cerveza y come nada.
Row 16: Me gustaría las precios cajas.
Row 17: Cruza a la derecha y sigue el toro resto.
Row 18: Ellos han terminado de asustar su apartamento.
Row 19: Me gustaría hacer más calor pronto.
Row 20: El niño que se murió, el gato, y está triste.
Row 21: Un amigo me cuidó a los niños.
Row 22: El gato negro fuera perseguido por el perro.
Row 23: Antes de ir a la tienda y limpiar el cuarto.
Row 24: La cantidad de personas que fuman es...
Row 25: Después de llegar a la casa, después del trabajo, comen la cena.
Row 26: El ladrón que atrapó con la policía era famoso.
```